

## ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

### ASSIGNMENT – 7

---

NAME – SUFIYA AFREEN

ENROLLMENT NO – 2403A51067

DATE – 2-9-26

---

Lab/Exp 6: Implementing Regularization: Dropout and L2 Weight Decay.

Kaggle Dataset Link: <https://www.kaggle.com/c/cifar-10>

Tasks:

1. Load and preprocess the CIFAR-10 dataset/example.
  2. Build the required PyTorch model/program.
  3. Train/execute the model.
  4. Evaluate using suitable metrics.
  5. Visualize results using plots where applicable.
  6. Write a brief conclusion comparing observations.
-

```
# STEP 1: IMPORT LIBRARIES
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# Check device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# =====
# STEP 2: LOAD CIFAR-10 DATASET
# =====

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        (0.5, 0.5, 0.5),
        (0.5, 0.5, 0.5)
    )
])

train_data = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_data = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

train_loader = torch.utils.data.DataLoader(
    train_data,
    batch_size=64,
    shuffle=True
)

test_loader = torch.utils.data.DataLoader(
    test_data,
    batch_size=64,
    shuffle=False
)
```

```

    ),
    print("Dataset loaded successfully!")
    print("Training images:", len(train_data))
    print("Testing images:", len(test_data))

# =====
# STEP 3: DISPLAY SAMPLE IMAGES
# =====

classes = [
    'airplane', 'automobile', 'bird', 'cat', 'deer',
    'dog', 'frog', 'horse', 'ship', 'truck'
]

images, labels = next(iter(train_loader))

plt.figure(figsize=(10, 5))

for i in range(8):
    plt.subplot(2, 4, i + 1)

    # Convert image for displaying
    img = images[i] / 2 + 0.5
    img = img.permute(1, 2, 0)

    plt.imshow(img)
    plt.title(classes[labels[i]])
    plt.axis("off")

plt.show()

# =====
# STEP 4: CREATE CNN MODEL
# =====

class CNN(nn.Module):

    def __init__(self, dropout=False):

        super(CNN, self).__init__()

        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)

        self.pool = nn.MaxPool2d(2, 2)

```



```
self.dropout = nn.Dropout(0.5)

self.fc1 = nn.Linear(128 * 4 * 4, 256)
self.fc2 = nn.Linear(256, 10)

self.use_dropout = dropout

def forward(self, x):

    x = torch.relu(self.conv1(x))
    x = self.pool(x)

    x = torch.relu(self.conv2(x))
    x = self.pool(x)

    x = torch.relu(self.conv3(x))
    x = self.pool(x)

    x = x.view(x.size(0), -1)

    x = torch.relu(self.fc1(x))

    if self.use_dropout:
        x = self.dropout(x)

    x = self.fc2(x)

    return x

# =====
# STEP 5: TRAINING FUNCTION
# =====

def train_model(model, optimizer, epochs=3):

    criterion = nn.CrossEntropyLoss()

    train_losses = []
    train_accuracies = []

    model.to(device)

    for epoch in range(epochs):

        model.train()

        running_loss = 0
        correct = 0
```

```

def test_model(model):

    model.eval()

    correct = 0
    total = 0

    with torch.no_grad():

        for images, labels in test_loader:

            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)

            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total

    return accuracy

# =====
# STEP 7: BASELINE MODEL
# =====

print("\n=====")
print("BASELINE MODEL")
print("=====")

baseline = CNN(dropout=False)

optimizer1 = optim.Adam(
    baseline.parameters(),
    lr=0.001
)

loss1, acc1 = train_model(
    baseline,
    optimizer1,
    epochs=3
)

test_acc1 = test_model(baseline)

```



```
running_loss = 0
correct = 0
total = 0

for images, labels in train_loader:

    images = images.to(device)
    labels = labels.to(device)

    # Reset gradients
    optimizer.zero_grad()

    # Forward pass
    outputs = model(images)

    # Calculate loss
    loss = criterion(outputs, labels)

    # Backpropagation
    loss.backward()

    # Update weights
    optimizer.step()

    running_loss += loss.item()

    _, predicted = torch.max(outputs, 1)

    total += labels.size(0)
    correct += (predicted == labels).sum().item()

loss_value = running_loss / len(train_loader)
accuracy = 100 * correct / total

train_losses.append(loss_value)
train_accuracies.append(accuracy)

print(
    "Epoch", epoch + 1,
    "Loss:", round(loss_value, 4),
    "Accuracy:", round(accuracy, 2), "%"
)

return train_losses, train_accuracies

# =====
# STEP 6: TESTING FUNCTION
# =====
```

```
*** Device: cpu
100%|██████████| 170M/170M [50:50<00:00, 55.9kB/s]
Dataset loaded successfully!
Training images: 50000
Testing images: 10000
```



=====

BASELINE MODEL

=====

Epoch 1 Loss: 1.3991 Accuracy: 49.06 %  
Epoch 2 Loss: 0.958 Accuracy: 66.21 %